test.ps1

```powershell
<#
.SYNOPSIS
    Enterprise System & Directory Analytics Engine (VRAM EDITION) - BACKGROUND LOAD
.DESCRIPTION
    Heavy JSON files load asynchronously so the menu appears immediately.
#>

[CmdletBinding()]
param(
    [Parameter(Position = 0, ValueFromPipeline = $true)]
    [ValidateNotNullOrEmpty()]
    [string]$Path = (Get-Location).Path,

    [Parameter(Position = 1)]
    [string]$Music,

    [Switch]$NoMusic,

    [ValidateRange(5, 500)]
    [int]$MaxDisplayRows = 50
)

Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'

# ---------- Crash Handler ----------
$script:ErrorLogFile = Join-Path $PSScriptRoot "error_log.txt"
function Write-CrashLog {
    param([string]$Message, [Exception]$Exception)
    $timestamp = Get-Date -Format "yyyy-MM-dd HH:mm:ss"
    $logEntry = "[$timestamp] $Message`n$($Exception.ToString())`n`n"
    Add-Content -Path $script:ErrorLogFile -Value $logEntry -Encoding UTF8
}

$global:ErrorActionPreference = 'Stop'
trap {
    Write-CrashLog -Message "Unhandled exception" -Exception $_
    Write-Host "`n[CRASH] An unexpected error occurred. Details logged to $script:ErrorLogFile" -ForegroundColor Red
    if ($script:musicPlayer) { Stop-AudioSession }
    exit 1
}
# ---------------------------------

$script:ENGINE_VERSION = "v10.0-VRAM-BGLOAD"
$script:EngineState = @{
    AudioProcess      = $null
    AudioPlaying      = $false
```

```
    AudioTrack        = "VOID / MUTED"
    DevMode           = $false
    FloatingBar       = $true
    NarratorData      = $null
    ConversationData  = $null
    SynonymData       = $null
    VRAMFile          = $null          # For catalog or other uses
    VRAMConvFile      = $null          # VRAM handle for conversation_commands.json
    VRAMConvIsArray   = $false
    VRAMConvMonolithicCache = $null
    VRAMSynonymFile   = $null
    LastScan          = $null
    ChatHistory       = @()
    FileCatalog       = @{}
    CodeCounter       = 0
    ConversationContext = @{
        topic        = $null
        lastIntent   = $null
        lastQuestion = $null
        userFrustration = 0
        sessionDepth = 0
        followUpCount = 0
    }
    UserProfile = @{
        name        = $null
        preferences = @{}
        interactionCount = 0
    }
    # Background load status
    LoadJob           = $null          # PowerShell job handle
    LoadComplete      = $false         # flag set when job finishes
    LoadError         = $null          # error message if job fails
}

# ================================================================================
# VIRTUAL RAM FILE SYSTEM (same as before, with fixes)
# ================================================================================

function New-VRAMFile {
    param([string]$FilePath)

    $vram = @{
        Path = $FilePath
        Size = (Get-Item $FilePath).Length
        Stream = $null
        Reader = $null
        BlockSize = 8192
        BlockCache = @{}
        Index = @{}          # key -> @{ Start, End, Name (optional) }
        TotalBlocks = 0
```

```
        IsObjectRoot = $false
    }

    $vram.Stream = [System.IO.File]::OpenRead($FilePath)
    $vram.Reader = New-Object System.IO.BinaryReader($vram.Stream)
    $vram.TotalBlocks = [math]::Ceiling($vram.Size / $vram.BlockSize)

    Write-Progress -Activity "VRAM Indexing" -Status "Scanning structure..." -
PercentComplete 0

    $buffer = New-Object byte[] 65536
    $position = 0
    $totalRead = 0
    $depth = 0
    $inString = $false
    $objectStart = 0
    $objectCount = 0
    $rootDepth = -1
    $expectPropName = $false
    $currentPropName = $null
    $propStart = 0
    $propDepth = 0
    $isRootObject = $false
    $foundFirstObject = $false
    $firstObjectStart = 0
    $firstObjectEnd = 0
    $firstObjectContent = $null

    $vram.Stream.Seek(0, [System.IO.SeekOrigin]::Begin) | Out-Null

    # First, peek to see if root is an object or array
    $peekBuffer = New-Object byte[] 1024
    $bytesRead = $vram.Stream.Read($peekBuffer, 0, 1024)
    $peekString = [System.Text.Encoding]::UTF8.GetString($peekBuffer, 0, $bytesRead)
    $trimmed = $peekString.TrimStart()
    $isObjectRoot = $trimmed.StartsWith('{')
    $vram.Stream.Seek(0, [System.IO.SeekOrigin]::Begin) | Out-Null

    while ($totalRead -lt $vram.Size) {
        $bytesRead = $vram.Stream.Read($buffer, 0, $buffer.Length)
        if ($bytesRead -eq 0) { break }

        for ($i = 0; $i -lt $bytesRead; $i++) {
            $byte = $buffer[$i]
            $char = [char]$byte
            $position++

            if ($char -eq '"' -and ($i -eq 0 -or $buffer[$i-1] -ne 92)) {
                $inString = -not $inString
            }
```

```
147
148             if (-not $inString) {
149                 if ($char -eq '{') {
150                     if ($depth -eq 0) {
151                         $objectStart = $position
152                         if (-not $foundFirstObject) {
153                             $firstObjectStart = $position
154                             $foundFirstObject = $true
155                         }
156                     }
157                     $depth++
158                     # if we are at root depth and we are in object mode, track property value
    start
159                     if ($depth -eq 2 -and $isObjectRoot -and $currentPropName) {
160                         $propStart = $position
161                         $propDepth = $depth
162                     }
163                 }
164                 if ($char -eq '}') {
165                     $depth--
166                     if ($depth -eq 0) {
167                         $objectCount++
168                         if ($objectCount -eq 1) {
169                             $firstObjectEnd = $position
170                         }
171                         # if we are in object mode and we just closed a property value
172                         if ($isObjectRoot -and $currentPropName -and $propDepth -eq $depth +
    1) {
173                             # record the property as an index entry
174                             $vram.Index["prop_$currentPropName"] = @{
175                                 Start = $propStart
176                                 End = $position
177                                 Name = $currentPropName
178                             }
179                             $currentPropName = $null
180                             $propDepth = 0
181                         } else {
182                             # normal object (array element)
183                             $vram.Index["obj_$objectCount"] = @{
184                                 Start = $objectStart
185                                 End = $position
186                             }
187                         }
188                     }
189                     if ($depth -eq 0) {
190                         $isRootObject = $false
191                     }
192                 }
193                 # Capture property name: when at depth 1 (inside root object) and see a '"'
    not in string
```

```
                if ($char -eq '"' -and $depth -eq 1 -and -not $inString -and $isObjectRoot)
{
                    # We'll capture the name when we see the closing quote
                    # For simplicity, we'll use a simpler approach: after the root object is
opened,
                    # we will not handle nested property names; we'll just treat the whole
root as one object.
                    # Better to use a JSON parser, but we'll fallback: if the file is a
single object,
                    # we will load it entirely on first use (which is still memory-efficient
for a single load).
                }
            }
        }

        $totalRead += $bytesRead
        $percent = [math]::Min(100, [int](($totalRead / $vram.Size) * 100))
        Write-Progress -Activity "VRAM Indexing" -Status "Scanning... $percent%" -
PercentComplete $percent
    }

    Write-Progress -Activity "VRAM Indexing" -Completed

    # If we only found one object and the root is an object, mark it as monolithic.
    if ($vram.Index.Count -eq 1 -and $isObjectRoot) {
        $vram.IsObjectRoot = $true
        Write-Host "  [VRAM] Detected monolithic object. Will parse entire file on first
use." -ForegroundColor Yellow
    } else {
        Write-Host "  [VRAM] Indexed $($vram.Index.Count) JSON objects." -ForegroundColor
Green
    }

    # Add methods
    $vram.GetObject = {
        param([string]$Key)
        if (-not $this.Index.ContainsKey($Key)) { return $null }
        $entry = $this.Index[$Key]
        $start = $entry.Start - 1
        $length = $entry.End - $entry.Start + 1
        $this.Stream.Seek($start, [System.IO.SeekOrigin]::Begin) | Out-Null
        $buffer = New-Object byte[] $length
        $this.Stream.Read($buffer, 0, $length) | Out-Null
        return [System.Text.Encoding]::UTF8.GetString($buffer)
    }

    $vram.GetKeys = { return $this.Index.Keys }
    $vram.GetCount = { return $this.Index.Count }
    $vram.Close = {
        if ($this.Reader) { $this.Reader.Close() }
        if ($this.Stream) { $this.Stream.Close() }
```

```
237         $this.BlockCache.Clear()
238     }
239 
240     return $vram
241 }
242 
243 function Open-VRAM {
244     param([string]$FilePath)
245     if (-not (Test-Path $FilePath)) {
246         Write-Host "  [VRAM] File not found: $FilePath" -ForegroundColor Red
247         return $null
248     }
249     $size = (Get-Item $FilePath).Length / 1MB
250     Write-Host "  [VRAM] Opening $([math]::Round($size,2)) MB file..." -ForegroundColor Cyan
251     return New-VRAMFile -FilePath $FilePath
252 }
253 
254 # ================================================================================
255 # AUDIO & SPEECH FUNCTIONS (unchanged)
256 # ================================================================================
257 
258 function Invoke-SystemSound {
259     param(
260         [ValidateSet('Beep', 'Success', 'Warning', 'Error', 'Toggle', 'Chat', 'Thinking',
    'TaskComplete')]
261         [string]$Type
262     )
263     if ($NoMusic) { return }
264     try {
265         switch ($Type) {
266             'Success' { [Console]::Beep(523, 80); [Console]::Beep(659, 80);
    [Console]::Beep(784, 100) }
267             'Warning' { [Console]::Beep(440, 120); [Console]::Beep(349, 120) }
268             'Error'   { [Console]::Beep(150, 200) }
269             'Toggle'  { [Console]::Beep(800, 50) }
270             'Chat'    { [Console]::Beep(600, 60); Start-Sleep -Milliseconds 50;
    [Console]::Beep(800, 60) }
271             'Thinking' { [Console]::Beep(300, 40); Start-Sleep -Milliseconds 30;
    [Console]::Beep(400, 40); Start-Sleep -Milliseconds 30; [Console]::Beep(500, 40) }
272             'TaskComplete' { [Console]::Beep(523, 80); [Console]::Beep(659, 80);
    [Console]::Beep(784, 100); Start-Sleep -Milliseconds 100; [Console]::Beep(1047, 150) }
273             Default   { [Console]::Beep(600, 60) }
274         }
275     } catch {}
276 }
277 
278 function Invoke-SpeechText {
279     param([string]$SpeechText, [int]$Rate = 0)
280     if ([string]::IsNullOrWhiteSpace($SpeechText)) { return }
281     try {
282         $voice = New-Object -ComObject SAPI.SpVoice
```

```
283         $voice.Volume = 100
284         $voice.Rate = $Rate
285         [void]$voice.Speak($SpeechText, 0)
286         [System.Runtime.InteropServices.Marshal]::ReleaseComObject($voice) | Out-Null
287     } catch {}
288 }
289
290 # ================================================================================
291 # DATA LOADERS (now used both synchronously and in background job)
292 # ================================================================================
293
294 function Import-NarratorData {
295     $scriptDir = if ($PSScriptRoot) { $PSScriptRoot } else { (Get-Location).Path }
296     $dataFile = Join-Path $scriptDir "narrator_data.json"
297     if (Test-Path -LiteralPath $dataFile) {
298         try {
299             $rawJson = Get-Content -LiteralPath $dataFile -Raw -Encoding UTF8
300             $script:EngineState.NarratorData = $rawJson | ConvertFrom-Json
301             Write-Host "  [NARRATOR] Loaded voice data library." -ForegroundColor Green
302         } catch {
303             Write-Host "  [NARRATOR ERROR] Failed to parse narrator_data.json: $_" -
ForegroundColor Red
304             Invoke-SystemSound -Type Error
305         }
306     } else {
307         Write-Host "  [NARRATOR WARNING] narrator_data.json not found." -ForegroundColor
Yellow
308     }
309 }
310
311 # This function is called by the background job (and also used synchronously if needed)
312 function Import-ConversationDataSync {
313     param([string]$ConvFile)
314     if (-not (Test-Path -LiteralPath $ConvFile)) { return $null }
315
316     $fileSize = (Get-Item $ConvFile).Length / 1MB
317     Write-Host "  [CHATBOT] File size: $([math]::Round($fileSize,2)) MB" -ForegroundColor
Gray
318
319     $useVRAM = $fileSize -gt 10
320     if ($useVRAM) {
321         Write-Host "  [CHATBOT] Large file. Using VRAM streaming..." -ForegroundColor Cyan
322         try {
323             $vram = Open-VRAM -FilePath $ConvFile
324             if ($null -eq $vram) {
325                 Write-Host "  [CHATBOT] VRAM failed. Falling back to standard loading." -
ForegroundColor Yellow
326             } else {
327                 # Store the VRAM handle; we will search on-demand.
328                 $result = @{
329                     ConversationData = $null
```

```
                    VRAMConvFile = $vram
                    VRAMConvIsArray = ($vram.GetCount() -gt 1)
                    VRAMConvMonolithicCache = $null
                }
                Write-Host "  [CHATBOT] VRAM ready for on-demand intent matching." -ForegroundColor Green
                return $result
            }
        } catch {
            Write-Host "  [CHATBOT] VRAM error: $_" -ForegroundColor Yellow
        }
    }

    # Fallback: standard loading for small files or if VRAM fails
    try {
        Write-Host "  Loading conversation database (standard mode)..." -ForegroundColor DarkGray
        $rawJson = Get-Content -LiteralPath $ConvFile -Raw -Encoding UTF8
        $convData = $rawJson | ConvertFrom-Json
        $intentCount = $convData.intents.PSObject.Properties.Name.Count
        Write-Host "  [CHATBOT] Loaded $intentCount intents into memory." -ForegroundColor Green
        return @{
            ConversationData = $convData
            VRAMConvFile = $null
            VRAMConvIsArray = $false
            VRAMConvMonolithicCache = $null
        }
    } catch {
        Write-Host "  [CHATBOT ERROR] Failed to parse: $_" -ForegroundColor Red
        return $null
    }
}

function Import-SynonymDataSync {
    param([string]$SynFile)
    if (-not (Test-Path -LiteralPath $SynFile)) { return $null }
    try {
        $fileSize = (Get-Item $SynFile).Length / 1MB
        Write-Host "  [SYNONYM] Loading $([math]::Round($fileSize,2)) MB file..." -ForegroundColor DarkGray

        # Use VRAM for large files, but if it's a single object, we might need to load it all.
        if ($fileSize -gt 10) {
            $vram = Open-VRAM -FilePath $SynFile
            if ($vram) {
                # Check if it's a single object
                if ($vram.GetCount() -eq 1) {
                    # Load the whole object (it might be a map)
                    $rootJson = $vram.GetObject("obj_1")
```

```
                    if ($rootJson) {
                        $synData = $rootJson | ConvertFrom-Json
                        Write-Host "  [SYNONYM] Loaded monolithic object via VRAM." -
ForegroundColor Green
                        $vram.Close()
                        return $synData
                    }
                } else {
                    # Multiple objects? unlikely for synonyms; we can combine
                    $allData = @{ synonym_map = @{} }
                    foreach ($key in $vram.GetKeys()) {
                        $obj = $vram.GetObject($key) | ConvertFrom-Json
                        if ($obj.PSObject.Properties) {
                            foreach ($prop in $obj.PSObject.Properties) {
                                $allData.synonym_map[$prop.Name] = $prop.Value
                            }
                        }
                    }
                    Write-Host "  [SYNONYM] Loaded combined objects via VRAM." -
ForegroundColor Green
                    $vram.Close()
                    return $allData
                }
            }
        }

        # Standard loading
        $rawJson = Get-Content -LiteralPath $SynFile -Raw -Encoding UTF8
        $synData = $rawJson | ConvertFrom-Json
        $synCount = $synData.synonym_map.PSObject.Properties.Name.Count
        Write-Host "  [SYNONYM] Loaded $synCount synonym groups." -ForegroundColor Green
        return $synData
    } catch {
        Write-Host "  [SYNONYM ERROR] Failed to parse synonyms.json: $_" -ForegroundColor
Red
        return $null
    }
}

# ================================================================================
# BACKGROUND LOADER JOB
# ================================================================================

function Start-BackgroundLoad {
    $scriptDir = if ($PSScriptRoot) { $PSScriptRoot } else { (Get-Location).Path }
    $convFile = Join-Path $scriptDir "conversation_commands.json"
    $synFile = Join-Path $scriptDir "synonyms.json"

    # Start the job
    $jobScript = {
        param($ConvPath, $SynPath)
```

```
424         # We need to include function definitions inside the job.
425         # We'll use a simplified load that doesn't depend on outer functions.
426         # Actually, we can just use the synchronous functions defined in the main script, but they won't be available in the job.
427         # Instead, we'll duplicate the loading logic inside the job script block.
428         # For brevity, we'll define a minimal version here.
429         # To keep it clean, we'll just run the synchronous functions if we can dot-source them, but we can't from a job.
430         # So we'll write a self-contained loader.
431
432         # --- VRAM functions (copied) ---
433         function New-VRAMFileJob {
434             param([string]$FilePath)
435             $vram = @{
436                 Path = $FilePath
437                 Size = (Get-Item $FilePath).Length
438                 Stream = $null
439                 Reader = $null
440                 BlockSize = 8192
441                 BlockCache = @{}
442                 Index = @{}
443                 TotalBlocks = 0
444                 IsObjectRoot = $false
445             }
446             $vram.Stream = [System.IO.File]::OpenRead($FilePath)
447             $vram.Reader = New-Object System.IO.BinaryReader($vram.Stream)
448             $vram.TotalBlocks = [math]::Ceiling($vram.Size / $vram.BlockSize)
449
450             $buffer = New-Object byte[] 65536
451             $position = 0
452             $totalRead = 0
453             $depth = 0
454             $inString = $false
455             $objectStart = 0
456             $objectCount = 0
457             $isRootObject = $false
458             $foundFirstObject = $false
459
460             $peekBuffer = New-Object byte[] 1024
461             $bytesRead = $vram.Stream.Read($peekBuffer, 0, 1024)
462             $peekString = [System.Text.Encoding]::UTF8.GetString($peekBuffer, 0, $bytesRead)
463             $trimmed = $peekString.TrimStart()
464             $isObjectRoot = $trimmed.StartsWith('{')
465             $vram.Stream.Seek(0, [System.IO.SeekOrigin]::Begin) | Out-Null
466
467             while ($totalRead -lt $vram.Size) {
468                 $bytesRead = $vram.Stream.Read($buffer, 0, $buffer.Length)
469                 if ($bytesRead -eq 0) { break }
470                 for ($i = 0; $i -lt $bytesRead; $i++) {
471                     $byte = $buffer[$i]
```

```
                    $char = [char]$byte
                    $position++
                    if ($char -eq '"' -and ($i -eq 0 -or $buffer[$i-1] -ne 92)) {
                        $inString = -not $inString
                    }
                    if (-not $inString) {
                        if ($char -eq '{') {
                            if ($depth -eq 0) { $objectStart = $position; if (-not $foundFirstObject) { $foundFirstObject = $true } }
                            $depth++
                        }
                        if ($char -eq '}') {
                            $depth--
                            if ($depth -eq 0) {
                                $objectCount++
                                $vram.Index["obj_$objectCount"] = @{ Start = $objectStart; End = $position }
                            }
                        }
                    }
                }
                $totalRead += $bytesRead
            }
            if ($vram.Index.Count -eq 1 -and $isObjectRoot) {
                $vram.IsObjectRoot = $true
            }
            $vram.GetObject = {
                param([string]$Key)
                if (-not $this.Index.ContainsKey($Key)) { return $null }
                $entry = $this.Index[$Key]
                $start = $entry.Start - 1
                $length = $entry.End - $entry.Start + 1
                $this.Stream.Seek($start, [System.IO.SeekOrigin]::Begin) | Out-Null
                $buffer = New-Object byte[] $length
                $this.Stream.Read($buffer, 0, $length) | Out-Null
                return [System.Text.Encoding]::UTF8.GetString($buffer)
            }
            $vram.GetKeys = { return $this.Index.Keys }
            $vram.GetCount = { return $this.Index.Count }
            $vram.Close = {
                if ($this.Reader) { $this.Reader.Close() }
                if ($this.Stream) { $this.Stream.Close() }
                $this.BlockCache.Clear()
            }
            return $vram
        }
        function Open-VRAMJob { param($FilePath) return New-VRAMFileJob -FilePath $FilePath }

        # --- Load conversation ---
        $convResult = $null
```

```
        if (Test-Path $ConvPath) {
            $fileSize = (Get-Item $ConvPath).Length / 1MB
            $useVRAM = $fileSize -gt 10
            if ($useVRAM) {
                try {
                    $vram = Open-VRAMJob -FilePath $ConvPath
                    if ($vram) {
                        $convResult = @{
                            ConversationData = $null
                            VRAMConvFile = $vram
                            VRAMConvIsArray = ($vram.GetCount() -gt 1)
                            VRAMConvMonolithicCache = $null
                        }
                    }
                } catch {}
            }
            if (-not $convResult) {
                try {
                    $rawJson = Get-Content -LiteralPath $ConvPath -Raw -Encoding UTF8
                    $convData = $rawJson | ConvertFrom-Json
                    $convResult = @{
                        ConversationData = $convData
                        VRAMConvFile = $null
                        VRAMConvIsArray = $false
                        VRAMConvMonolithicCache = $null
                    }
                } catch {}
            }
        }

        # --- Load synonyms ---
        $synResult = $null
        if (Test-Path $SynPath) {
            try {
                $fileSize = (Get-Item $SynPath).Length / 1MB
                if ($fileSize -gt 10) {
                    $vram = Open-VRAMJob -FilePath $SynPath
                    if ($vram) {
                        if ($vram.GetCount() -eq 1) {
                            $rootJson = $vram.GetObject("obj_1")
                            if ($rootJson) {
                                $synResult = $rootJson | ConvertFrom-Json
                                $vram.Close()
                            }
                        } else {
                            $allData = @{ synonym_map = @{} }
                            foreach ($key in $vram.GetKeys()) {
                                $obj = $vram.GetObject($key) | ConvertFrom-Json
                                if ($obj.PSObject.Properties) {
                                    foreach ($prop in $obj.PSObject.Properties) {
```

```
                                    $allData.synonym_map[$prop.Name] = $prop.Value
                                }
                            }
                        }
                        $synResult = $allData
                        $vram.Close()
                    }
                }
            }
            if (-not $synResult) {
                $rawJson = Get-Content -LiteralPath $SynPath -Raw -Encoding UTF8
                $synResult = $rawJson | ConvertFrom-Json
            }
        } catch {}
    }

    # Return both results
    return @{ Conversation = $convResult; Synonyms = $synResult }
}

    # Start the job
    $script:EngineState.LoadJob = Start-Job -ScriptBlock $jobScript -ArgumentList $convFile, $synFile
    Write-Host "  [BACKGROUND] Started loading conversation and synonyms in background." -ForegroundColor Cyan
}

function Wait-ForBackgroundLoad {
    # If job is not running, return
    if (-not $script:EngineState.LoadJob) { return }
    if ($script:EngineState.LoadComplete) { return }

    Write-Host "`n  [BACKGROUND] Waiting for data to load... (this may take a while)" -ForegroundColor Yellow
    $job = $script:EngineState.LoadJob
    # Wait for job with progress
    while ($job.State -eq 'Running') {
        Write-Host "  [BACKGROUND] Still loading... (use Ctrl+C to cancel and use fallback)" -ForegroundColor DarkGray
        Start-Sleep -Seconds 2
    }
    # Get results
    if ($job.State -eq 'Completed') {
        $result = Receive-Job -Job $job
        if ($result) {
            # Update EngineState
            if ($result.Conversation) {
                $conv = $result.Conversation
                $script:EngineState.ConversationData = $conv.ConversationData
                $script:EngineState.VRAMConvFile = $conv.VRAMConvFile
                $script:EngineState.VRAMConvIsArray = $conv.VRAMConvIsArray
```

```
                $script:EngineState.VRAMConvMonolithicCache = $conv.VRAMConvMonolithicCache
                Write-Host "  [BACKGROUND] Conversation data loaded." -ForegroundColor Green
            }
            if ($result.Synonyms) {
                $script:EngineState.SynonymData = $result.Synonyms
                Write-Host "  [BACKGROUND] Synonyms loaded." -ForegroundColor Green
            }
            $script:EngineState.LoadComplete = $true
        } else {
            Write-Host "  [BACKGROUND] Load returned no data." -ForegroundColor Yellow
            $script:EngineState.LoadComplete = $true
        }
    } else {
        Write-Host "  [BACKGROUND] Job failed: $($job.State)" -ForegroundColor Red
        $script:EngineState.LoadError = "Background job failed."
        $script:EngineState.LoadComplete = $true
    }
    Remove-Job -Job $job -Force
    $script:EngineState.LoadJob = $null
}

# ================================================================================
# VLC AUDIO SUBSYSTEM (unchanged)
# ================================================================================

function Get-VlcPath {
    $possiblePaths = @(
        "C:\Program Files\VideoLAN\VLC\vlc.exe",
        "C:\Program Files (x86)\VideoLAN\VLC\vlc.exe",
        "$env:LOCALAPPDATA\Programs\VideoLAN\VLC\vlc.exe"
    )
    foreach ($path in $possiblePaths) {
        if (Test-Path -LiteralPath $path) { return $path }
    }
    $vlcCmd = Get-Command "vlc.exe" -ErrorAction SilentlyContinue
    if ($vlcCmd) { return $vlcCmd.Source }
    return $null
}

function Start-AudioSession {
    param([string]$FilePath)
    if ($NoMusic) { return }
    if ([string]::IsNullOrWhiteSpace($FilePath)) { return }
    if (-not (Test-Path -LiteralPath $FilePath -PathType Leaf)) {
        Write-Host "  [AUDIO] Asset unmapped: '$FilePath'" -ForegroundColor Yellow
        Start-Sleep -Seconds 1
        return
    }
    $vlcExe = Get-VlcPath
    if ($null -eq $vlcExe) {
```

```
667         Write-Host "  [AUDIO WARNING] VLC Media Player not found." -ForegroundColor Yellow
668         Start-Sleep -Seconds 2
669         return
670     }
671     $resolvedPath = (Resolve-Path -LiteralPath $FilePath).Path
672     try {
673         $vlcArgs = @("-I", "dummy", "--loop", "--gain=0.65", "--no-video",
    "`"$resolvedPath`"")
674         $procInfo = New-Object System.Diagnostics.ProcessStartInfo
675         $procInfo.FileName = $vlcExe
676         $procInfo.Arguments = $vlcArgs -join " "
677         $procInfo.WindowStyle = [System.Diagnostics.ProcessWindowStyle]::Hidden
678         $procInfo.CreateNoWindow = $true
679         $procInfo.UseShellExecute = $false
680         $process = [System.Diagnostics.Process]::Start($procInfo)
681         $script:EngineState.AudioProcess = $process
682         $script:EngineState.AudioPlaying = $true
683         $script:EngineState.AudioTrack = Split-Path -Leaf $resolvedPath
684     } catch {
685         Write-Host "  [AUDIO EXCEPTION] VLC failed to start: $_" -ForegroundColor DarkRed
686     }
687 }
688
689 function Stop-AudioSession {
690     if ($null -ne $script:EngineState.AudioProcess) {
691         try {
692             if (-not $script:EngineState.AudioProcess.HasExited) {
693                 $script:EngineState.AudioProcess.Kill()
694                 $script:EngineState.AudioProcess.Dispose()
695             }
696         } catch {}
697         $script:EngineState.AudioProcess = $null
698         $script:EngineState.AudioPlaying = $false
699     }
700 }
701
702 # ==============================================================================
703 # UI COMPONENTS (unchanged)
704 # ==============================================================================
705
706 function Show-FloatingBar {
707     if (-not $script:EngineState.FloatingBar) { return }
708     $statusStr = if ($script:EngineState.AudioPlaying) { "♫ [VLC] " +
    $script:EngineState.AudioTrack } else { "x ACOUSTIC SILENCE" }
709     $devStr    = if ($script:EngineState.DevMode) { "DIAG: ACTIVE" } else { "DIAG: STABLE" }
710     Write-Host "
    ┌──────────────────────────────────────────────────────────────────────────┐" -
    ForegroundColor DarkGray
711     Write-Host ("  │ CODESPACE: {0,-32} │ HUD: {1,-12} │ ENGINE: {2,-5} │" -f $statusStr,
    $devStr, $script:ENGINE_VERSION) -ForegroundColor Cyan
```

```
    Write-Host "
╚═══════════════════════════════════════════════════════════════════════════════╝" -
ForegroundColor DarkGray
    Write-Host ""
}

function Show-DevDashboard {
    if (-not $script:EngineState.DevMode) { return }
    $mem = [System.GC]::GetTotalMemory($false) / 1MB
    $proc = [System.Diagnostics.Process]::GetCurrentProcess()
    Write-Host "  ╔══ [ SYSTEM DIAGNOSTICS // Telemetry Core ]
═══════════════════════════╗" -ForegroundColor DarkMagenta
    Write-Host ("  ╟  > Shell Architecture : {0,-44} ╢" -f ("PowerShell v" +
$PSVersionTable.PSVersion)) -ForegroundColor Magenta
    Write-Host ("  ╟  > Process ID         : {0,-44} ╢" -f $proc.Id) -ForegroundColor
Magenta
    Write-Host ("  ╟  > Managed Heap Memory: {0,-44} ╢" -f ("{0:N2} MB" -f $mem)) -
ForegroundColor Magenta
    Write-Host ("  ╟  > Physical WorkingSet: {0,-44} ╢" -f ("{0:N2} MB" -f
($proc.WorkingSet64 / 1MB))) -ForegroundColor Magenta
    Write-Host "
╚═══════════════════════════════════════════════════════════════════════════════╝" -
ForegroundColor DarkMagenta
    Write-Host ""
}

# ================================================================================
# SYNONYM REPLACEMENT ENGINE (unchanged)
# ================================================================================

function Invoke-SynonymReplacement {
    param([string]$Text, [float]$ReplaceProbability = 0.15)

    if ($null -eq $script:EngineState.SynonymData) { return $Text }
    $synMap = $script:EngineState.SynonymData.synonym_map
    if ($null -eq $synMap) { return $Text }

    $words = $Text -split ' '
    $newWords = @()
    foreach ($word in $words) {
        $cleanWord = $word -replace '[^a-zA-Z]', ''
        $punctuation = $word -replace '[a-zA-Z]', ''
        if ($cleanWord.Length -gt 2 -and $synMap.PSObject.Properties.Name -contains
$cleanWord) {
            if ((Get-Random -Minimum 0 -Maximum 1) -lt $ReplaceProbability) {
                $synonyms = $synMap.$cleanWord
                if ($synonyms -and $synonyms.Count -gt 0) {
                    $newWord = $synonyms | Get-Random
                    $newWords += $newWord + $punctuation
                    continue
                }
```

```
            }
        }
        $newWords += $word
    }
    return $newWords -join ' '
}

# ================================================================================
# NARRATOR FUNCTIONS (unchanged)
# ================================================================================

function Get-FormattedByteSize {
    param([long]$Bytes)
    if ($Bytes -ge 1GB) { return "{0:N2} GB" -f ($Bytes / 1GB) }
    if ($Bytes -ge 1MB) { return "{0:N2} MB" -f ($Bytes / 1MB) }
    if ($Bytes -ge 1KB) { return "{0:N2} KB" -f ($Bytes / 1KB) }
    return "$Bytes B"
}

function New-HumanNarration {
    param([System.IO.FileInfo]$File, [int]$LineCount)
    $ext = $File.Extension.ToLower()
    if ([string]::IsNullOrWhiteSpace($ext)) { $ext = "default" }
    $data = $script:EngineState.NarratorData
    if ($null -eq $data) {
        $script = "Now inspecting $($File.Name). It has $LineCount lines of code and data."
        return @{ Fact = "Every byte is stored securely."; SpeechScript = $script; ExtensionDesc = "Generic file node" }
    }
    $intro = $data.intro_phrases | Get-Random
    $extDesc = if ($data.extension_narratives.PSObject.Properties[$ext]) { $data.extension_narratives.$ext } else { $data.extension_narratives.default }
    $sizeCategory = if ($File.Length -lt 100) { "pico" } elseif ($File.Length -lt 1KB) { "nano" } elseif ($File.Length -lt 10KB) { "micro" } elseif ($File.Length -lt 100KB) { "compact" } elseif ($File.Length -lt 1MB) { "moderate" } elseif ($File.Length -lt 10MB) { "standard" } elseif ($File.Length -lt 100MB) { "heavy" } else { "very_large" }
    $sizeDesc = $data.size_descriptions.$sizeCategory
    $triviaList = if ($data.trivia_database.PSObject.Properties[$ext]) { $data.trivia_database.$ext } else { $data.trivia_database.default }
    $selectedFact = $triviaList | Get-Random
    $outro = $data.outro_phrases | Get-Random

    $baseScript = "$intro This file is named $($File.Name). $extDesc $sizeDesc It contains $LineCount lines in total. $selectedFact $outro"
    $variedScript = Invoke-SynonymReplacement -Text $baseScript -ReplaceProbability 0.2

    return @{
        Fact          = $selectedFact
        SpeechScript  = $variedScript
        ExtensionDesc = $extDesc
    }
```

```
}

# ============================================================================
# FILE CODE ALLOTMENT SYSTEM (unchanged)
# ============================================================================

function Invoke-FileCodeAllotment {
    param([string]$TargetDirectory, [string]$CatalogPath = "file_catalog.json")

    Write-Host ""
    Write-Host "
╔═══════════════════════════════════════════════════════════════════════════╗" -ForegroundColor DarkBlue
    Write-Host "  ║                    ▓▓▓▓▓  FILE CODE ALLOTMENT  ▓▓▓▓▓                    ║" -ForegroundColor Cyan
    Write-Host "
╚═══════════════════════════════════════════════════════════════════════════╝" -ForegroundColor DarkBlue
    Write-Host ""
    Write-Host "  Assigning unique codes to every file in the directory..." -ForegroundColor Gray
    Write-Host ""

    $resolvedDirectory = (Resolve-Path -LiteralPath $TargetDirectory).Path.TrimEnd('\', '/')
    $files = Get-ChildItem -LiteralPath $resolvedDirectory -File -Recurse -Force -ErrorAction SilentlyContinue

    if ($files.Count -eq 0) {
        Write-Host "  [ERROR] No files found in the directory." -ForegroundColor Red
        Invoke-SystemSound -Type Error
        return
    }

    $catalog = @{}
    if (Test-Path -LiteralPath $CatalogPath) {
        try {
            $json = Get-Content -LiteralPath $CatalogPath -Raw -Encoding UTF8
            $catalog = $json | ConvertFrom-Json
            Write-Host "  [INFO] Loaded existing catalog with $($catalog.PSObject.Properties.Name.Count) entries." -ForegroundColor Green
        } catch {}
    }

    Write-Host "  Processing $($files.Count) files..." -ForegroundColor Gray

    $spinners = @('⠋', '⠙', '⠹', '⠸', '⠼', '⠴', '⠦', '⠧', '⠇', '⠏')
    $extMap = @{}
    $assignedCount = 0
    $updatedCatalog = @{}
    $rootPath = $resolvedDirectory

```

```
    foreach ($file in $files) {
        $spinIdx = $assignedCount % $spinners.Length
        Write-Host "`r   [$($spinners[$spinIdx])] Assigning codes to files... ($($assignedCount + 1)/$($files.Count))" -NoNewline -ForegroundColor Cyan

        $relativePath = $file.FullName.Substring($rootPath.Length).TrimStart('\', '/')
        $ext = if ($file.Extension) { $file.Extension.TrimStart('.').ToUpper() } else { "NOEXT" }

        $prefix = if ($ext -eq "NOEXT") { "NEX" } elseif ($ext.Length -ge 3) { $ext.Substring(0, 3) } else { $ext.PadRight(3, 'X') }

        $hash = Get-FileHash -Path $file.FullName -Algorithm MD5 -ErrorAction SilentlyContinue
        $hashShort = if ($hash) { $hash.Hash.Substring(0, 6) } else { "000000" }

        $code = "ECHO-$prefix-$($assignedCount.ToString('D6'))-$hashShort"

        $existingCode = $null
        if ($catalog.PSObject.Properties.Name) {
            foreach ($key in $catalog.PSObject.Properties.Name) {
                if ($catalog.$key.path -eq $relativePath) {
                    $existingCode = $key
                    break
                }
            }
        }

        if ($existingCode) { $code = $existingCode }

        $lineCount = 0
        try {
            $reader = New-Object System.IO.StreamReader($file.FullName, [System.Text.Encoding]::UTF8, $true)
            while ($reader.ReadLine() -ne $null) { $lineCount++ }
            $reader.Close()
        } catch { $lineCount = 0 }

        $updatedCatalog[$code] = @{
            path = $relativePath
            filename = $file.Name
            extension = $ext
            size_bytes = $file.Length
            size_human = Get-FormattedByteSize $file.Length
            line_count = $lineCount
            last_modified = $file.LastWriteTime.ToString("yyyy-MM-dd HH:mm:ss")
            hash_md5 = if ($hash) { $hash.Hash } else { "N/A" }
            code = $code
        }

        if (-not $extMap.ContainsKey($ext)) { $extMap[$ext] = 0 }
```

```
        $extMap[$ext]++
        $assignedCount++
    }

    Write-Host "`r   [✓] Assigned codes to $assignedCount files.
" -ForegroundColor Green

    $catalogJson = $updatedCatalog | ConvertTo-Json -Depth 10
    $utf8NoBom = New-Object System.Text.UTF8Encoding $false
    [System.IO.File]::WriteAllText($CatalogPath, $catalogJson, $utf8NoBom)

    Write-Host "  [OK] Catalog saved to: $CatalogPath" -ForegroundColor Green
    Write-Host "  [INFO] Total files coded: $assignedCount" -ForegroundColor Cyan
    Write-Host "  [INFO] Extension breakdown:" -ForegroundColor Gray
    foreach ($ext in $extMap.Keys | Sort-Object) {
        Write-Host ("    ├─ {0,-15} : {1,5} files" -f $ext, $extMap[$ext]) -ForegroundColor
DarkGray
    }

    Invoke-SystemSound -Type TaskComplete
    return $updatedCatalog
}

function Invoke-CodeSearch {
    param([string]$CatalogPath = "file_catalog.json", [string]$SearchTerm)

    if (-not (Test-Path -LiteralPath $CatalogPath)) {
        Write-Host "  [ERROR] Catalog not found: $CatalogPath" -ForegroundColor Red
        return
    }

    $json = Get-Content -LiteralPath $CatalogPath -Raw -Encoding UTF8
    $catalog = $json | ConvertFrom-Json

    Write-Host ""
    Write-Host "  ── [ CODE SEARCH RESULTS ] ──────────────────────────────────────────────"
-ForegroundColor Cyan

    $found = 0
    foreach ($key in $catalog.PSObject.Properties.Name) {
        $entry = $catalog.$key
        if ($key -match $SearchTerm -or $entry.path -match $SearchTerm -or $entry.filename -
match $SearchTerm) {
            Write-Host ("  CODE: {0}" -f $key) -ForegroundColor Yellow
            Write-Host ("    PATH: {0}" -f $entry.path) -ForegroundColor Gray
            Write-Host ("    SIZE: {0}  |  LINES: {1}" -f $entry.size_human,
$entry.line_count) -ForegroundColor DarkCyan
            Write-Host ""
            $found++
        }
    }
```

```
932
933     if ($found -eq 0) {
934         Write-Host "  No entries found matching: $SearchTerm" -ForegroundColor Yellow
935     } else {
936         Write-Host "  Found $found entries." -ForegroundColor Green
937     }
938 }
939
940 function Invoke-CatalogReport {
941     param([string]$CatalogPath = "file_catalog.json")
942
943     if (-not (Test-Path -LiteralPath $CatalogPath)) {
944         Write-Host "  [ERROR] Catalog not found: $CatalogPath" -ForegroundColor Red
945         return
946     }
947
948     $json = Get-Content -LiteralPath $CatalogPath -Raw -Encoding UTF8
949     $catalog = $json | ConvertFrom-Json
950
951     $totalFiles = $catalog.PSObject.Properties.Name.Count
952     $totalSize = 0
953     $totalLines = 0
954     $extStats = @{}
955
956     foreach ($key in $catalog.PSObject.Properties.Name) {
957         $entry = $catalog.$key
958         $totalSize += $entry.size_bytes
959         $totalLines += $entry.line_count
960         $ext = $entry.extension
961         if (-not $extStats.ContainsKey($ext)) { $extStats[$ext] = 0 }
962         $extStats[$ext]++
963     }
964
965     Write-Host ""
966     Write-Host "
    ╔══════════════════════════════════════════════════════════════════════════════╗" -
    ForegroundColor DarkBlue
967     Write-Host "  ║                          FILE CATALOG REPORT
    ║" -ForegroundColor Cyan
968     Write-Host "
    ╚══════════════════════════════════════════════════════════════════════════════╝" -
    ForegroundColor DarkBlue
969     Write-Host ""
970     Write-Host ("  • Total Files : {0,8:N0}" -f $totalFiles) -ForegroundColor White
971     Write-Host ("  • Total Size  : {0,8}" -f (Get-FormattedByteSize $totalSize)) -
    ForegroundColor White
972     Write-Host ("  • Total Lines : {0,8:N0}" -f $totalLines) -ForegroundColor White
973     Write-Host ""
974     Write-Host "  ── [ EXTENSION BREAKDOWN ] ──────────────────────────────────────────"
    -ForegroundColor DarkBlue
975     foreach ($ext in $extStats.Keys | Sort-Object) {
```

```
976         Write-Host ("    ├─ {0,-15} : {1,5} files" -f $ext, $extStats[$ext]) -
ForegroundColor Gray
977     }
978     Write-Host ""
979 }
980
981 # ================================================================================
982 # SYSTEM ANALYTICS ENGINE (unchanged)
983 # ================================================================================
984
985 function Invoke-SystemScanEngine {
986     param([string]$TargetDirectory)
987     $resolvedDirectory = (Resolve-Path -LiteralPath $TargetDirectory).Path.TrimEnd('\', '/')
988     $stats = [PSCustomObject]@{ TotalFolders = 0; EmptyFolders = 0; TotalFiles = 0;
EmptyFiles = 0; ActiveFiles = 0; TotalLines = 0; ScanDuration = 0.0; TopExtensions = @{};
LargestFile = $null }
989     $sw = [System.Diagnostics.Stopwatch]::StartNew()
990     $spinners = @('⠋', '⠙', '⠹', '⠸', '⠼', '⠴', '⠦', '⠧', '⠇', '⠏')
991     $spinIdx = 0
992     Write-Host "   [$($spinners[$spinIdx])] Traversing directory tree & indexing nodes..." -
NoNewline -ForegroundColor Cyan
993     $allItems = Get-ChildItem -LiteralPath $resolvedDirectory -Recurse -Force -ErrorAction
SilentlyContinue
994     $dirs = [System.Collections.Generic.List[System.IO.DirectoryInfo]]::new()
995     $files = [System.Collections.Generic.List[System.IO.FileInfo]]::new()
996     $parentNodes =
[System.Collections.Generic.HashSet[string]]::new([System.StringComparer]::OrdinalIgnoreCase)
997     foreach ($item in $allItems) {
998         if ($item.PSIsContainer) { $dirs.Add($item); if ($null -ne $item.Parent) {
[void]$parentNodes.Add($item.Parent.FullName) } }
999         else { $files.Add($item); [void]$parentNodes.Add($item.DirectoryName) }
1000     }
1001     $stats.TotalFolders = $dirs.Count
1002     foreach ($d in $dirs) { if (-not $parentNodes.Contains($d.FullName)) {
$stats.EmptyFolders++ } }
1003     $stats.TotalFiles = $files.Count
1004     $extTracker = @{}
1005     $largestSize = -1
1006     $currentFileIdx = 0
1007     foreach ($f in $files) {
1008         $currentFileIdx++
1009         if ($currentFileIdx % 25 -eq 0) {
1010             $spinIdx = ($spinIdx + 1) % $spinners.Length
1011             Write-Host "`r   [$($spinners[$spinIdx])] Analyzing files ($currentFileIdx /
$($files.Count))..." -NoNewline -ForegroundColor Cyan
1012         }
1013         $ext = if ($f.Extension) { $f.Extension.ToLower() } else { "[no extension]" }
1014         if (-not $extTracker.ContainsKey($ext)) { $extTracker[$ext] = 0 }
1015         $extTracker[$ext]++
1016         if ($f.Length -gt $largestSize) { $largestSize = $f.Length; $stats.LargestFile = $f
}
```

```
        if ($f.Length -eq 0) { $stats.EmptyFiles++; continue }
        $stats.ActiveFiles++
        $lineCount = 0
        try {
            $stream = [System.IO.StreamReader]::new($f.FullName, [System.Text.Encoding]::UTF8, $true)
            while ($null -ne $stream.ReadLine()) { $lineCount++ }
            $stream.Dispose()
        } catch { $lineCount = 0 }
        $stats.TotalLines += $lineCount
    }
    $sw.Stop()
    Write-Host "`r   [✓] Scan completed successfully in $($sw.Elapsed.TotalSeconds.ToString('N2'))s              " -ForegroundColor Green
    $stats.ScanDuration = $sw.Elapsed.TotalSeconds
    $stats.TopExtensions = $extTracker
    $script:EngineState.LastScan = $stats
    return $stats
}

# ================================================================================
# RANDOM FILE SPEECH (unchanged, but ensures synonyms are loaded if needed)
# ================================================================================

function Invoke-RandomFileSpeech {
    param([string]$TargetDirectory)
    # Ensure synonyms are loaded (wait for background job if needed)
    if (-not $script:EngineState.SynonymData -and -not $script:EngineState.LoadComplete) {
        Write-Host "  [NARRATOR] Synonyms not yet loaded. Waiting for background load..." -ForegroundColor Yellow
        Wait-ForBackgroundLoad
    }
    # If still not loaded, we proceed without synonyms (will work fine)

    $spinners = @('⠋', '⠙', '⠹', '⠸', '⠼', '⠴', '⠦', '⠧', '⠇', '⠏')
    for ($i = 0; $i -lt 5; $i++) {
        Write-Host "`r   [$($spinners[$i % $spinners.Length])] Querying file clusters & selecting random node..." -NoNewline -ForegroundColor Magenta
        Start-Sleep -Milliseconds 60
    }
    $rawFiles = Get-ChildItem -LiteralPath $TargetDirectory -File -Recurse -Force -ErrorAction SilentlyContinue
    $allFiles = @($rawFiles | Where-Object { $_ -ne $null })
    if ($allFiles.Count -eq 0) {
        Write-Host "`r   [!] Target directory contains no readable files.                " -ForegroundColor Yellow
        Invoke-SystemSound -Type Warning
        return
    }
    $randomFile = $allFiles | Get-Random
    $rootPath = (Resolve-Path -LiteralPath $TargetDirectory).Path.TrimEnd('\', '/')
```

```
1062      $relativePath = $randomFile.FullName.Substring($rootPath.Length).TrimStart('\', '/')
1063      $rawLines = @()
1064      try { $content = Get-Content -LiteralPath $randomFile.FullName -ErrorAction Stop;
          $rawLines = @($content) }
1065      catch {
1066          Write-Host "`r   [ERROR] Unable to read file stream: $_                          " -
          ForegroundColor Red
1067          Invoke-SystemSound -Type Error; return
1068      }
1069      $totalLineCount = $rawLines.Count
1070      $formattedSize = Get-FormattedByteSize $randomFile.Length
1071      $narration = New-HumanNarration -File $randomFile -LineCount $totalLineCount
1072      Write-Host "`r   [✓] Node isolated successfully: $($randomFile.Name)                  "
          -ForegroundColor Green
1073      Write-Host ""
1074      Write-Host "  ┌─ [ TARGET FILE METRICS ]
          ──────────────────────────────────────────────┐" -ForegroundColor DarkBlue
1075      Write-Host ("  │ Target File Node     ==> {0,-46} │" -f $randomFile.Name) -
          ForegroundColor Cyan
1076      Write-Host ("  │ Relative Path        ==> {0,-46} │" -f ($relativePath | Select-Object -
          First 46)) -ForegroundColor Gray
1077      Write-Host ("  │ Extension Class      ==> {0,-46} │" -f $narration.ExtensionDesc) -
          ForegroundColor Yellow
1078      Write-Host ("  │ File Size            ==> {0,-46} │" -f $formattedSize) -ForegroundColor
          DarkCyan
1079      Write-Host ("  │ Total Lines          ==> {0,-46} │" -f $totalLineCount) -
          ForegroundColor White
1080      Write-Host "
          └──────────────────────────────────────────────────────────────────────┘" -ForegroundColor
          DarkBlue
1081      Write-Host ""
1082      Write-Host "  ── [ CODE PREVIEW ] ──────────────────────────────────────────────────" -
          ForegroundColor DarkBlue
1083      $previewLines = $rawLines | Select-Object -First 12
1084      foreach ($line in $previewLines) {
1085          $trimmed = if ($line -and $line.Length -gt 70) { $line.Substring(0, 67) + "..." }
          else { $line }
1086          Write-Host "    | " -ForegroundColor DarkBlue -NoNewline
1087          Write-Host $trimmed -ForegroundColor Gray
1088          [System.Threading.Thread]::Sleep(10)
1089      }
1090      Write-Host ""
1091      Write-Host "  ── [ HUMAN-LIKE NARRATOR OUTPUT ] ────────────────────────────────────" -
          ForegroundColor DarkMagenta
1092      Write-Host ("  * SCRIPT : {0}" -f $narration.SpeechScript) -ForegroundColor Magenta
1093      Write-Host ""
1094      Invoke-SystemSound -Type Success
1095      Write-Host "  [VOICE SYNTH] Narrator speaking..." -ForegroundColor Cyan
1096      Invoke-SpeechText -SpeechText $narration.SpeechScript
1097  }
1098
1099  # ================================================================================
```

```powershell
# CHATBOT ENGINE – with lazy loading and VRAM search (unchanged logic)
# ============================================================================

function Find-MatchingIntentVRAM {
    param([string]$UserInput)
    $vram = $script:EngineState.VRAMConvFile
    if (-not $vram) { return $null }
    $bestScore = 0
    $bestResponses = $null

    if ($vram.IsObjectRoot) {
        if ($null -eq $script:EngineState.VRAMConvMonolithicCache) {
            $key = $vram.GetKeys() | Select-Object -First 1
            if ($key) {
                $json = $vram.GetObject($key)
                try {
                    $script:EngineState.VRAMConvMonolithicCache = $json | ConvertFrom-Json
                } catch {
                    return $null
                }
            }
        }
        $obj = $script:EngineState.VRAMConvMonolithicCache
        if ($obj.intents) {
            foreach ($intentName in $obj.intents.PSObject.Properties.Name) {
                $intent = $obj.intents.$intentName
                if ($intent.patterns) {
                    foreach ($pattern in $intent.patterns) {
                        if ($UserInput -match $pattern) {
                            if ($intent.responses) {
                                return $intent.responses | Get-Random
                            }
                        }
                    }
                }
            }
        }
        return $null
    }

    $keys = $vram.GetKeys()
    foreach ($key in $keys) {
        $objJson = $vram.GetObject($key)
        try {
            $obj = $objJson | ConvertFrom-Json
        } catch { continue }
        if ($obj.PSObject.Properties.Name -contains "patterns") {
            foreach ($pattern in $obj.patterns) {
                if ($UserInput -match $pattern) {
                    if ($obj.responses) {
```

```
                        return $obj.responses | Get-Random
                    }
                }
            }
        } elseif ($obj.PSObject.Properties.Name -contains "intents") {
            $intents = $obj.intents
            foreach ($intentName in $intents.PSObject.Properties.Name) {
                $intent = $intents.$intentName
                if ($intent.patterns) {
                    foreach ($pattern in $intent.patterns) {
                        if ($UserInput -match $pattern) {
                            if ($intent.responses) {
                                return $intent.responses | Get-Random
                            }
                        }
                    }
                }
            }
        }
    }
    return $null
}

function Invoke-SentimentAnalysis {
    param([string]$Text)
    $positiveWords = @('thanks', 'thank', 'great', 'awesome', 'good', 'nice', 'wonderful', 'excellent', 'love', 'happy', 'enjoy', 'amazing', 'perfect', 'fantastic', 'brilliant', 'glad', 'appreciate')
    $negativeWords = @('bad', 'terrible', 'awful', 'hate', 'dislike', 'annoy', 'frustrating', 'useless', 'broken', 'fail', 'error', 'bug', 'crash', 'stupid', 'worst', 'horrible', 'sad', 'angry')
    $curiousWords = @('why', 'how', 'what', 'when', 'where', 'who', 'which', 'explain', 'tell', 'show', 'understand', 'curious', 'wonder', 'question')
    $words = $Text.ToLower() -split ' ' | Where-Object { $_.Length -gt 2 }
    $posScore = 0; $negScore = 0; $curScore = 0
    foreach ($w in $words) {
        if ($positiveWords -contains $w) { $posScore++ }
        if ($negativeWords -contains $w) { $negScore++ }
        if ($curiousWords -contains $w) { $curScore++ }
    }
    if ($posScore -gt $negScore -and $posScore -gt 0) { return "positive" }
    if ($negScore -gt $posScore -and $negScore -gt 0) { return "negative" }
    if ($curScore -gt 2) { return "curious" }
    return "neutral"
}

function Test-FollowUp {
    param([string]$Text)
    $followUpPatterns = @("tell me more", "explain more", "continue", "go on", "and then", "what else", "more about", "elaborate", "further", "next", "after that", "what happened",
```

```
"then what", "so what", "why is that", "how so", "interesting", "hmm", "i see", "okay",
"right", "got it", "makes sense", "understood")
1194    $lower = $Text.ToLower()
1195    foreach ($pattern in $followUpPatterns) {
1196        if ($lower -match $pattern) { return $true }
1197    }
1198    return $false
1199 }
1200
1201 function Detect-UserName {
1202    param([string]$Text)
1203    $patterns = @("my name is (\w+)", "i am (\w+)", "call me (\w+)", "i'm (\w+)")
1204    foreach ($pattern in $patterns) {
1205        if ($Text -match $pattern) { return $matches[1] }
1206    }
1207    return $null
1208 }
1209
1210 function New-FallbackResponse {
1211    param([string]$Sentiment)
1212    $fallbacks = @{
1213        "positive" = @("I'm glad you're enjoying the conversation!", "That's great to
hear!", "Wonderful! What else would you like to discuss?")
1214        "negative" = @("I sense some frustration. Let me try to help more clearly.", "I
apologize if I'm not being clear enough.", "I want to help. Can you tell me more?")
1215        "curious" = @("That's a great question! Let me think about it.", "I love your
curiosity!", "Excellent question! Here's what I know.")
1216        "default" = @("That's an interesting topic. Can you tell me more?", "I want to
understand better. Could you explain?", "Hmm, that's a complex question.")
1217    }
1218    $list = if ($fallbacks.ContainsKey($Sentiment)) { $fallbacks[$Sentiment] } else {
$fallbacks["default"] }
1219    return $list | Get-Random
1220 }
1221
1222 function Invoke-Chatbot {
1223    # Ensure conversation data is loaded (wait for background job if needed)
1224    if (-not $script:EngineState.ConversationData -and -not $script:EngineState.VRAMConvFile
-and -not $script:EngineState.LoadComplete) {
1225        Write-Host "  [CHATBOT] Conversation data not yet loaded. Waiting for background
load..." -ForegroundColor Yellow
1226        Wait-ForBackgroundLoad
1227    }
1228    # If still not loaded, try to load synchronously as fallback
1229    if (-not $script:EngineState.ConversationData -and -not
$script:EngineState.VRAMConvFile) {
1230        Write-Host "  [CHATBOT] Attempting to load conversation data synchronously..." -
ForegroundColor Yellow
1231        $scriptDir = if ($PSScriptRoot) { $PSScriptRoot } else { (Get-Location).Path }
1232        $convFile = Join-Path $scriptDir "conversation_commands.json"
1233        $result = Import-ConversationDataSync -ConvFile $convFile
```

```
1234        if ($result) {
1235            $script:EngineState.ConversationData = $result.ConversationData
1236            $script:EngineState.VRAMConvFile = $result.VRAMConvFile
1237            $script:EngineState.VRAMConvIsArray = $result.VRAMConvIsArray
1238            $script:EngineState.VRAMConvMonolithicCache = $result.VRAMConvMonolithicCache
1239        }
1240    }
1241    # If still nothing, error out
1242    $convData = $script:EngineState.ConversationData
1243    $vram = $script:EngineState.VRAMConvFile
1244    if (-not $convData -and -not $vram) {
1245        Write-Host "  [CHATBOT ERROR] No conversation data available." -ForegroundColor Red
1246        Start-Sleep -Seconds 3
1247        return
1248    }
1249
1250    # Now the rest of chatbot is exactly as before
1251    Write-Host ""
1252    Write-Host "
    ╔══════════════════════════════════════════════════════════════════════════════╗" -
    ForegroundColor DarkBlue
1253    Write-Host "  ║          ▄▄▄▄▄  ADVANCED AI CHATBOT  ▄▄▄▄▄
    ║" -ForegroundColor Cyan
1254    Write-Host "  ║          Echo Engine v10.0 — with Context, Memory & Personality
    ║" -ForegroundColor Gray
1255    Write-Host "
    ╚══════════════════════════════════════════════════════════════════════════════╝" -
    ForegroundColor DarkBlue
1256    Write-Host ""
1257    Write-Host "  I'm your AI assistant with contextual memory and personality!" -
    ForegroundColor Green
1258    Write-Host "  Commands: 'exit' or 'quit' to return, 'clear' to clear history, 'help' for
    info" -ForegroundColor Gray
1259    Write-Host "  You can also ask me about the file catalog and codes!" -ForegroundColor
    Gray
1260    Write-Host ""
1261
1262    $chatActive = $true
1263    $script:EngineState.ChatHistory = @()
1264    $ctx = $script:EngineState.ConversationContext
1265
1266    while ($chatActive) {
1267        if ($script:EngineState.ChatHistory.Count -gt 0) {
1268            Write-Host "  ── [Conversation History]
    ──────────────────────────────────────────" -ForegroundColor DarkGray
1269            $startIdx = [Math]::Max(0, $script:EngineState.ChatHistory.Count - 10)
1270            for ($i = $startIdx; $i -lt $script:EngineState.ChatHistory.Count; $i++) {
1271                $entry = $script:EngineState.ChatHistory[$i]
1272                $prefixStr = if ($entry.type -eq "user") { "  You: " } else { "  Bot: " }
1273                $color = if ($entry.type -eq "user") { "Cyan" } else { "Magenta" }
1274                Write-Host $prefixStr -NoNewline -ForegroundColor $color
```

```
1275                Write-Host $entry.message -ForegroundColor Gray
1276            }
1277            Write-Host ""
1278        }
1279
1280        Write-Host "  You: " -NoNewline -ForegroundColor Cyan
1281        $userInput = Read-Host
1282        $userInput = $userInput.Trim()
1283
1284        if ($userInput -eq "" -or $userInput -eq "exit" -or $userInput -eq "quit") {
1285            $chatActive = $false
1286            Write-Host "  [CHATBOT] Session ended." -ForegroundColor Gray
1287            Invoke-SystemSound -Type Beep
1288            break
1289        }
1290
1291        if ($userInput -eq "clear") {
1292            $script:EngineState.ChatHistory = @()
1293            $ctx.topic = $null; $ctx.lastIntent = $null; $ctx.lastQuestion = $null
1294            $ctx.followUpCount = 0; $ctx.userFrustration = 0; $ctx.sessionDepth = 0
1295            Clear-Host; Show-FloatingBar
1296            Write-Host "  [CHATBOT] Chat history and context cleared." -ForegroundColor Gray
1297            continue
1298        }
1299
1300        if ($userInput -eq "help") {
1301            Write-Host ""
1302            Write-Host "  ── [CHATBOT HELP] ──────────────────────────────────────────────────────────────────────────────────────────────────────────────" -ForegroundColor Cyan
1303            Write-Host "  I can talk about:"
1304            Write-Host "    • Myself (who I am, what I do)"
1305            Write-Host "    • File operations (scan, analyze, read files)"
1306            Write-Host "    • System status (memory, CPU, uptime)"
1307            Write-Host "    • File codes and catalog (ask about codes)"
1308            Write-Host "    • Technology (programming, AI, data)"
1309            Write-Host "    • Commands: exit, clear, help"
1310            Write-Host ""
1311            Invoke-SystemSound -Type Chat
1312            continue
1313        }
1314
1315        $detectedName = Detect-UserName -Text $userInput
1316        if ($detectedName) {
1317            $script:EngineState.UserProfile.name = $detectedName
1318            Write-Host "  [CHATBOT] Name recognized: $detectedName" -ForegroundColor Green
1319        }
1320
1321        $script:EngineState.ChatHistory += @{ type = "user"; message = $userInput }
1322        $script:EngineState.UserProfile.interactionCount++
1323        $ctx.sessionDepth++
```

```
1324          $sentiment = Invoke-SentimentAnalysis -Text $userInput
1325          if ($sentiment -eq "negative") { $ctx.userFrustration++ }
1326          $isFollowUp = Test-FollowUp -Text $userInput
1327          if ($isFollowUp) { $ctx.followUpCount++ }
1328
1329          # --- Intent matching ---
1330          $response = $null
1331
1332          if ($vram) {
1333              $response = Find-MatchingIntentVRAM -UserInput $userInput
1334          } elseif ($convData) {
1335              $intents = $convData.intents
1336              $matchedIntent = $null; $bestScore = 0
1337              foreach ($intentName in $intents.PSObject.Properties.Name) {
1338                  $intent = $intents.$intentName
1339                  foreach ($pattern in $intent.patterns) {
1340                      $patternWords = $pattern -split " " | Where-Object { $_.Length -gt 2 }
1341                      $matches = 0; $exactMatches = 0
1342                      foreach ($pw in $patternWords) {
1343                          $pwLower = $pw.ToLower()
1344                          if ($userInput.ToLower() -match $pwLower) {
1345                              $matches++
1346                              if ($userInput.ToLower() -match "\b$pwLower\b") {
      $exactMatches++ }
1347                          }
1348                      }
1349                      $score = if ($patternWords.Count -gt 0) { ($matches /
      $patternWords.Count) * 0.6 + ($exactMatches / $patternWords.Count) * 0.4 } else { 0 }
1350                      if ($isFollowUp -and $intentName -eq $ctx.lastIntent) { $score += 0.3 }
1351                      if ($score -gt $bestScore) { $bestScore = $score; $matchedIntent =
      $intent }
1352                  }
1353              }
1354              if ($null -ne $matchedIntent -and $bestScore -gt 0.15) {
1355                  $responses = $matchedIntent.responses
1356                  if ($responses -and $responses.Count -gt 0) {
1357                      $response = $responses | Get-Random
1358                      $ctx.lastIntent = $matchedIntent.PSObject.Properties.Name[0]
1359                  }
1360              }
1361          }
1362
1363          if (-not $response) {
1364              if ($ctx.userFrustration -gt 3) {
1365                  $response = "I apologize for the confusion. What specifically would you like
      to know about?"
1366                  $ctx.userFrustration = 0
1367              } elseif ($ctx.followUpCount -gt 2 -and $ctx.lastQuestion) {
1368                  $response = "You seem really interested in this topic! Let me dive deeper. "
      + $ctx.lastQuestion
1369                  $ctx.followUpCount = 0
```

```
            } else {
                $response = New-FallbackResponse -Sentiment $sentiment
            }
            if ($userInput -match "can you|will you|could you") { $response += " Of course! I'm here to help." }
        }

        if ($ctx.userFrustration -gt 2) {
            $response = "I want to make sure I'm helping. " + $response + " Would you like me to explain something differently?"
            $ctx.userFrustration = 0
        }

        if ($script:EngineState.UserProfile.name) {
            $name = $script:EngineState.UserProfile.name
            if (-not $response.StartsWith("I")) { $response = "$name, " + $response }
        }

        if ($userInput -match "what is|how do|why does|can you|tell me") {
            $ctx.lastQuestion = $userInput
        }

        Invoke-SystemSound -Type Thinking
        Write-Host "  Bot: " -NoNewline -ForegroundColor Magenta
        $thinkingChars = @('●', 'o', '●', 'o', '●')
        foreach ($t in $thinkingChars) {
            Write-Host $t -NoNewline -ForegroundColor DarkMagenta
            Start-Sleep -Milliseconds 60
            Write-Host "`b" -NoNewline
        }
        Start-Sleep -Milliseconds 100
        Write-Host "  " -NoNewline

        $emotionIcon = switch ($sentiment) { "positive" { "😊" } "negative" { "😳" } "curious" { "🤔" } default { "💬" } }
        Write-Host $emotionIcon -NoNewline -ForegroundColor Yellow
        Write-Host " " -NoNewline
        Write-Host $response -ForegroundColor White

        Invoke-SystemSound -Type Chat
        try { Invoke-SpeechText -SpeechText $response } catch {}

        $script:EngineState.ChatHistory += @{ type = "bot"; message = $response }
        if ($ctx.sessionDepth -gt 20) { $ctx.sessionDepth = 0; $ctx.followUpCount = 0 }
        Write-Host ""
        Start-Sleep -Milliseconds 200
    }
}

# ============================================================================
# MAIN CONTROL LOOP
```

```
# ================================================================================

try {
    # Load only the small narrator data immediately
    Import-NarratorData

    # Start background job for heavy data
    Start-BackgroundLoad

    # Optionally start audio
    $scriptDir = if ($PSScriptRoot) { $PSScriptRoot } else { (Get-Location).Path }
    $audioFile = if ($Music) { $Music } else { Join-Path $scriptDir "bgmusic.mp3" }
    Start-AudioSession -FilePath $audioFile

    $running = $true
    while ($running) {
        Clear-Host
        Write-Host ""
        Write-Host "
╔══════════════════════════════════════════════════════════════════════════════╗" -
ForegroundColor DarkBlue
        Write-Host "  ║          INTERSTELLAR QUANTUM ANALYTICS COMMAND CENTER v10.0 VRAM
║" -ForegroundColor Cyan
        Write-Host "  ║          Echo Engine — Virtual RAM File System for 1GB+ Files
║" -ForegroundColor Gray
        Write-Host "
╚══════════════════════════════════════════════════════════════════════════════╝" -
ForegroundColor DarkBlue
        Write-Host ""

        Show-FloatingBar
        Show-DevDashboard

        # Show load status
        if (-not $script:EngineState.LoadComplete) {
            Write-Host "  [⏳] Background loading in progress..." -ForegroundColor Yellow
        } else {
            Write-Host "  [✅] Data loaded" -ForegroundColor Green
        }

        Write-Host "   [1] Engage System Analytics Engine" -ForegroundColor Cyan
        Write-Host "   [2] Toggle Diagnostics HUD" -ForegroundColor White
        Write-Host "   [3] Toggle Status Telemetry Bar" -ForegroundColor White
        Write-Host "   [4] Inspect & Narrate Random File (with Synonyms)" -ForegroundColor
Magenta
        Write-Host "   [5] Change Target Directory Path" -ForegroundColor White
        Write-Host "   [6] 🤖 AI Chatbot Interface (VRAM Enhanced)" -ForegroundColor Green
        Write-Host "   [7] 📋 File Code Allotment (Assign Unique Codes)" -ForegroundColor
Yellow
        Write-Host "   [8] 🔍 Search File Catalog by Code/Name" -ForegroundColor Yellow
        Write-Host "   [9] 📊 Generate File Catalog Report" -ForegroundColor Yellow
```

```
        Write-Host "   [10] Exit Engine" -ForegroundColor DarkRed
        Write-Host ""
        Write-Host "   Active Target Path: " -NoNewline -ForegroundColor Gray
        Write-Host $Path -ForegroundColor Cyan
        Write-Host ""
        Write-Host "   VRAM Status: " -NoNewline
        if ($script:EngineState.VRAMConvFile) {
            Write-Host "✅ Active (conversation_commands.json)" -ForegroundColor Green
        } else {
            Write-Host "❌ Inactive" -ForegroundColor Red
        }
        Write-Host ""

        $choice = Read-Host "   Command Input [1-10]"

        switch ($choice) {
            '1' {
                Invoke-SystemSound -Type Beep
                Clear-Host
                Show-FloatingBar
                Write-Host "  [+] Executing directory tree analysis..." -ForegroundColor Cyan
                $scanStats = Invoke-SystemScanEngine -TargetDirectory $Path
                Clear-Host
                Show-FloatingBar
                Write-Host "
╔═══════════════════════════════════════════════════════════════════════════════╗" -
ForegroundColor DarkBlue
                Write-Host "  ║                      ANALYTICS TELEMETRY METRICS
║" -ForegroundColor Cyan
                Write-Host "
╚═══════════════════════════════════════════════════════════════════════════════╝" -
ForegroundColor DarkBlue
                Write-Host ""
                Write-Host ("   • Total Folders Scanned : {0:N0}" -f $scanStats.TotalFolders)
-ForegroundColor White
                Write-Host ("   • Empty Directories     : {0:N0}" -f $scanStats.EmptyFolders)
-ForegroundColor DarkGray
                Write-Host ("   • Total Files Scanned   : {0:N0}" -f $scanStats.TotalFiles)
-ForegroundColor White
                Write-Host ("   • Active Files (Non-0)  : {0:N0}" -f $scanStats.ActiveFiles)
-ForegroundColor DarkCyan
                Write-Host ("   • Total Lines Analyzed  : {0:N0}" -f $scanStats.TotalLines)
-ForegroundColor Magenta
                Write-Host ("   • Analytics Duration    : {0:N2}s" -f
$scanStats.ScanDuration) -ForegroundColor Green
                if ($null -ne $scanStats.LargestFile) {
                    Write-Host ("   • Largest Node Detected : {0} ({1})" -f
$scanStats.LargestFile.Name, (Get-FormattedByteSize $scanStats.LargestFile.Length)) -
ForegroundColor Yellow
                }
                Write-Host ""
```

```
1499                Write-Host "   ── [ FILE TYPE DISTRIBUTION ] ──────────────────────────────" -ForegroundColor DarkBlue
1500                $topExts = $scanStats.TopExtensions.GetEnumerator() | Sort-Object Value -Descending | Select-Object -First 5
1501                foreach ($ext in $topExts) {
1502                    Write-Host ("     ├─ {0,-15} : {1,5} file(s)" -f $ext.Key, $ext.Value) -ForegroundColor Gray
1503                }
1504                Invoke-SystemSound -Type Success
1505                Write-Host ""
1506                Write-Host "   Press ENTER to return to dashboard..." -ForegroundColor Cyan
1507                [void](Read-Host)
1508            }
1509            '2' { $script:EngineState.DevMode = -not $script:EngineState.DevMode; Invoke-SystemSound -Type Toggle }
1510            '3' { $script:EngineState.FloatingBar = -not $script:EngineState.FloatingBar; Invoke-SystemSound -Type Toggle }
1511            '4' {
1512                Invoke-RandomFileSpeech -TargetDirectory $Path
1513                Write-Host ""
1514                Write-Host "   Press ENTER to return to dashboard..." -ForegroundColor Cyan
1515                [void](Read-Host)
1516            }
1517            '5' {
1518                Write-Host ""
1519                $newPath = Read-Host "   Enter new path"
1520                if (Test-Path -LiteralPath $newPath -PathType Container) {
1521                    $Path = $newPath
1522                    Invoke-SystemSound -Type Success
1523                } else {
1524                    Write-Host "   [ERROR] Path does not exist." -ForegroundColor Red
1525                    Invoke-SystemSound -Type Error
1526                    Start-Sleep -Seconds 2
1527                }
1528            }
1529            '6' {
1530                Clear-Host
1531                Show-FloatingBar
1532                Invoke-Chatbot
1533                Clear-Host
1534            }
1535            '7' {
1536                Clear-Host
1537                Show-FloatingBar
1538                $catalog = Invoke-FileCodeAllotment -TargetDirectory $Path -CatalogPath "file_catalog.json"
1539                Write-Host ""
1540                Write-Host "   Press ENTER to return to dashboard..." -ForegroundColor Cyan
1541                [void](Read-Host)
1542            }
1543            '8' {
```

```
                Write-Host ""
                $searchTerm = Read-Host "   Enter code or filename to search"
                Invoke-CodeSearch -CatalogPath "file_catalog.json" -SearchTerm $searchTerm
                Write-Host ""
                Write-Host "   Press ENTER to return to dashboard..." -ForegroundColor Cyan
                [void](Read-Host)
            }
            '9' {
                Clear-Host
                Show-FloatingBar
                Invoke-CatalogReport -CatalogPath "file_catalog.json"
                Write-Host ""
                Write-Host "   Press ENTER to return to dashboard..." -ForegroundColor Cyan
                [void](Read-Host)
            }
            '10' { $running = $false }
            Default {
                Invoke-SystemSound -Type Warning
                Write-Host "  [WARNING] Invalid option. Please choose 1-10." -ForegroundColor
Yellow
                Start-Sleep -Seconds 1
            }
        }
    }
} catch {
    Write-CrashLog -Message "Fatal error in main loop" -Exception $_
    Write-Host ""
    Write-Host "  [ENGINE FATAL EXCEPTION] :: $_" -ForegroundColor Red
    Invoke-SystemSound -Type Error
} finally {
    # Clean up
    if ($script:EngineState.VRAMConvFile) { $script:EngineState.VRAMConvFile.Close() }
    if ($script:EngineState.VRAMSynonymFile) { $script:EngineState.VRAMSynonymFile.Close() }
    Stop-AudioSession
    Write-Host ""
    Write-Host "  Session terminated safely." -ForegroundColor White
}
```